

Exceptional Control Flow III

Outline

- Reaping Child Processes
- Putting Processes to Sleep
- Loading and Running Programs
- Suggested reading 8.4

Recap: Process

- Process
 - Shell
 - Multi-Process applications
 - Modularity, Security, Fault tolerance
- Interface
 - Fork
 - Exit
 - Getpid/Getppid

Recap: Three states of a process

- Running
 - The process is either **executing** on the CPU
 - or is **waiting** to be executed
 - and will eventually be scheduled
- Stopped (blocked)
 - The execution of the process is **suspended**
 - and will not be scheduled
- Terminated
 - The process is **stopped** permanently

Reap Child Processes

Zombie

- The kernel does not remove a terminated process from the system immediately after the process terminates
- The process is kept around in a terminated state until it is **reaped** by its parent



Last words

Zombie: Last words

- When the parent reaps the terminated child
 - the kernel passes the child's exit status to the parent
 - and then discards the terminated process, at which point it ceases to exist
- A terminated process that has not yet been reaped is called a **zombie**

Zombie: Who recycle

- If the parent process
 - terminates without reaping its zombie children
 - the kernel arranges for the `init` process to reap them
- The `init` process
 - has a PID of 1
 - and is created by the kernel during system initialization

Zombie: Do the recycle by yourself

- Long-running programs such as shells or servers should always reap their zombie children
- Even though zombies are not running, they still consume system memory resources

Waitpid Function

```
#include <sys/types.h>
```

```
#include <sys/wait.h>
```

```
pid_t waitpid(pid_t pid, int *status, int options);
```

```
pid_t wait(int *status);
```

Returns: PID of child if OK, 0 (if WNOHANG) or -1 on error

Waitpid Function

```
pid_t waitpid(pid_t pid, int *status, int options);
```

- `pid > 0`
 - The wait set is the **singleton** child process
 - whose process ID is equal to **pid**
- `pid = -1`
 - The wait set consists of **all** of the parent's child processes

Waitpid Function

- The `waitpid` function
 - If the calling process has **no** children
 - returns -1
 - sets `errno` to `ECHILD`
 - If interrupted by a **signal**
 - returns -1
 - sets `errno` to `EINTR`

Waitpid Function

```
pid_t waitpid(pid_t pid, int *status, int options);
```

- options = 0
 - waitpid suspends execution of the calling process until a child process in its **wait-set** terminates
 - If a process in the wait set has already terminated at the time of the call, then waitpid returns **immediately**
 - waitpid returns the PID of the terminated child that caused waitpid to return
 - the terminated child is removed from the system

Waitpid Function

```
pid_t waitpid(pid_t pid, int *status, int options);
```

- options
 - WNOHANG: Return **immediately** (with a return value of 0) if none of the child processes in the waiting set has terminated yet
 - WUNTRACED: Suspended execution of the calling process until a child process terminated or **stopped**

Waitpid Function

```
pid_t waitpid(pid_t pid, int *status, int options);
```

- Options (**composable**)
 - WUNTRACED | WNOHANG :
 - Return immediately (with a return value of 0) if none of the children in the waiting set has stopped or terminated
 - Or return a value equal to the PID of one of the stopped or terminated children

Waitpid Function

```
pid_t waitpid(pid_t pid, int *status, int options);
```

- Checking the exit status of a reaped child
 - The `status` argument is non-NULL
 - `waitpid` encodes status information about the child
 - that caused the return in the `status` argument
 - The `wait.h` include file defines several macros
 - for interpreting the `status` argument

Waitpid Function

```
pid_t waitpid(pid_t pid, int *status, int options);
```

- **WIFEXITED(status)**
 - Returns true if the child terminated normally
 - via a call to `exit` (e.g., return from main)
- **WEXITSTATUS(status)**
 - Returns the exit status of a normally terminated child
 - This status is only defined if **WIFEXITED** returned true

Waitpid Function

```
pid_t waitpid(pid_t pid, int *status, int options);
```

- **WIFSIGNALED(status)**
 - Returns true if the child process terminated because of a signal that was not caught
- **WTERMSIG(status)**
 - Returns the number of the signal that caused the child process to terminate
 - This status is only defined if **WIFSIGNALED** returned true

Waitpid Function

```
pid_t waitpid(pid_t pid, int *status, int options);
```

- **WIFSTOPPED(status)**
 - Returns true if the child that caused the return is currently stopped
- **WSTOPSIG(status)**
 - Returns the number of the signal that caused the child to stop
 - This status is only defined if **WIFSTOPPED** returned true

Waitpid Function (Nondeterministic)

```
1 #include "csapp.h"
2 #define N 2
3
4 int main()
5 {
6     int status, i;
7     pid_t pid;
8
9     /* Parent creates N children */
10    for (i = 0; i < N; i++)
11        if ((pid = Fork()) == 0) /* child */
12            exit(100+i);
13
```

Waitpid Function (Nondeterministic)

```
14  /* Parent reaps N chds. in no particular order */
15  while ((pid = waitpid(-1, &status, 0)) > 0) {
16      if (WIFEXITED(status))
17          printf("child %d terminated normally with exit
18                  status=%d\n", pid, WEXITSTATUS(status));
19      else
20          printf("child %d terminated abnormally\n", pid);
21  }
22
23  /* The only normal term. is if there no more chds. */
24  if (errno != ECHILD)
25      unix_error("waitpid error");
26
27  exit(0);
28}
```

Waitpid Function (Nondeterministic)

```
unix>./waitpid1
```

```
child 22966 terminated normally with exit status=100
```

```
child 22967 terminated normally with exit status=101
```

- There is no reaping order
 - Nondeterministic behavior for concurrency

Waitpid Function (Deterministic)

```
1 #include "csapp.h"
2 #define N 2
3
4 int main()
5 {
6     int status, i;
7     pid_t pid[N+1], retpid;
8
9     /* Parent creates N children */
10    for (i = 0; i < N; i++)
11        if ((pid[i] = Fork()) == 0) /* Child */
12            exit(100+i);
13
14    /* Parent reaps N children in order */
15    i = 0;
```

Waitpid Function (Deterministic)

```
15 while ((retpid = waitpid(pid[i++], &status, 0)) > 0) {
16     if (WIFEXITED(status))
17         printf("child %d terminated normally with exit
18                 status=%d\n", retpid, WEXITSTATUS(status));
19     else
20         printf("child %d terminated abnormally\n", retpid);
21 }
22
23
24 /* The only normal term. is if there are no more chds. */
25 if (errno != ECHILD)
26     unix_error("waitpid error");
27
28 exit(0);
29 }
```


SideBar: Putting Process to Sleep

```
#include <unistd.h>
```

```
unsigned int sleep(unsigned int secs);
```

Returns: seconds left to sleep

```
int pause(void);
```

Always returns -1

- Suspends a process for some period of time
- Returns zero if the requested amount of time has elapsed
- Returns the number of seconds still left to sleep otherwise

Load and Run Programs

Loading and Running

```
#include <unistd.h>
```

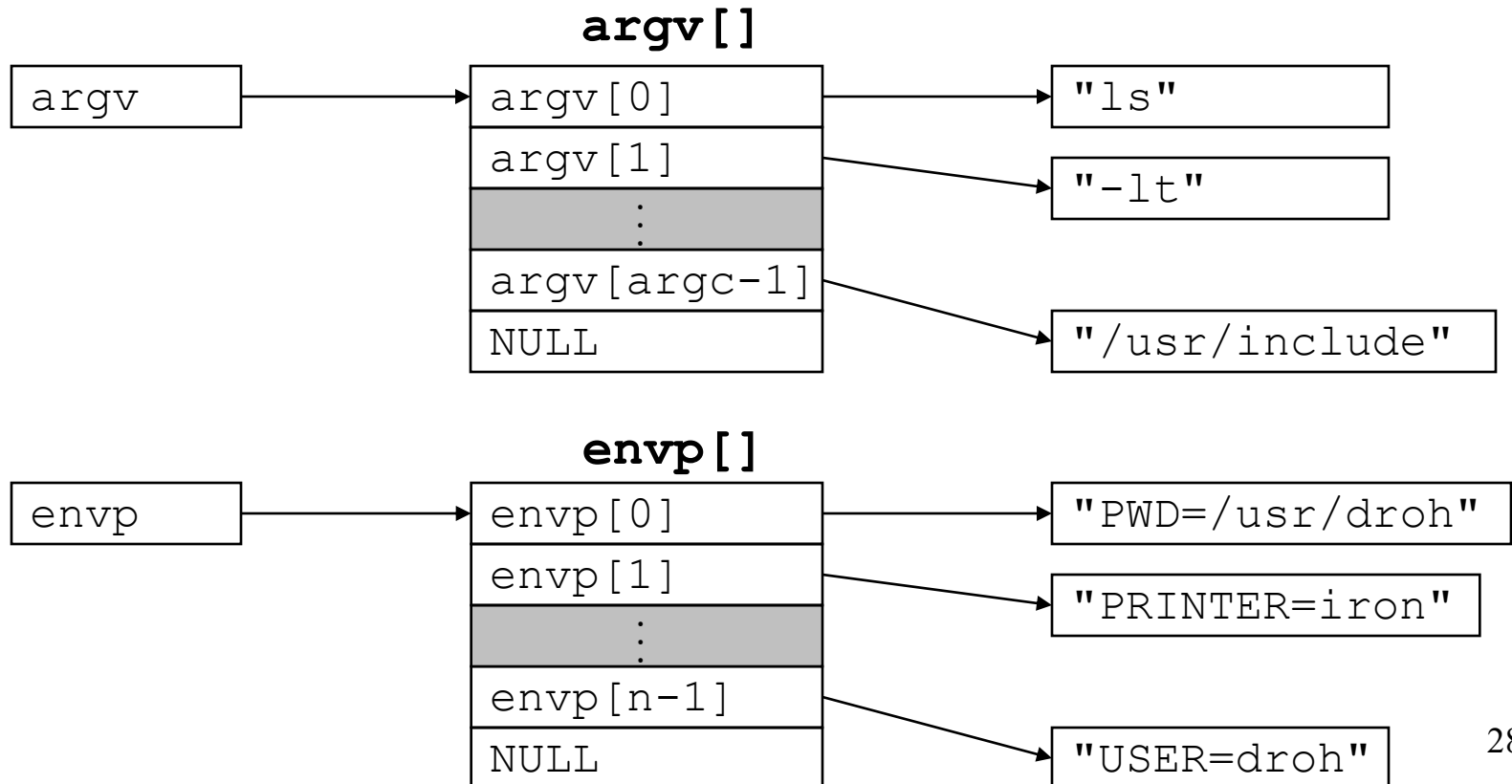
```
int execve(const char *filename, const char *argv[],  
           const char *envp[]);
```

does not return if OK, returns -1 on error

- Loads and runs
 - the executable object file `filename`
 - with the argument list `argv`
 - the environment variable list `envp`
 - returns to the calling program only if there is an error
 - such as not being able to find `filename`
- The `execve` is called **once** and **never** returns

Loading and Running

```
int execve(const char *filename, const char *argv[],  
           const char *envp[]);
```



The user stack when a new program starts

0x7fffffff

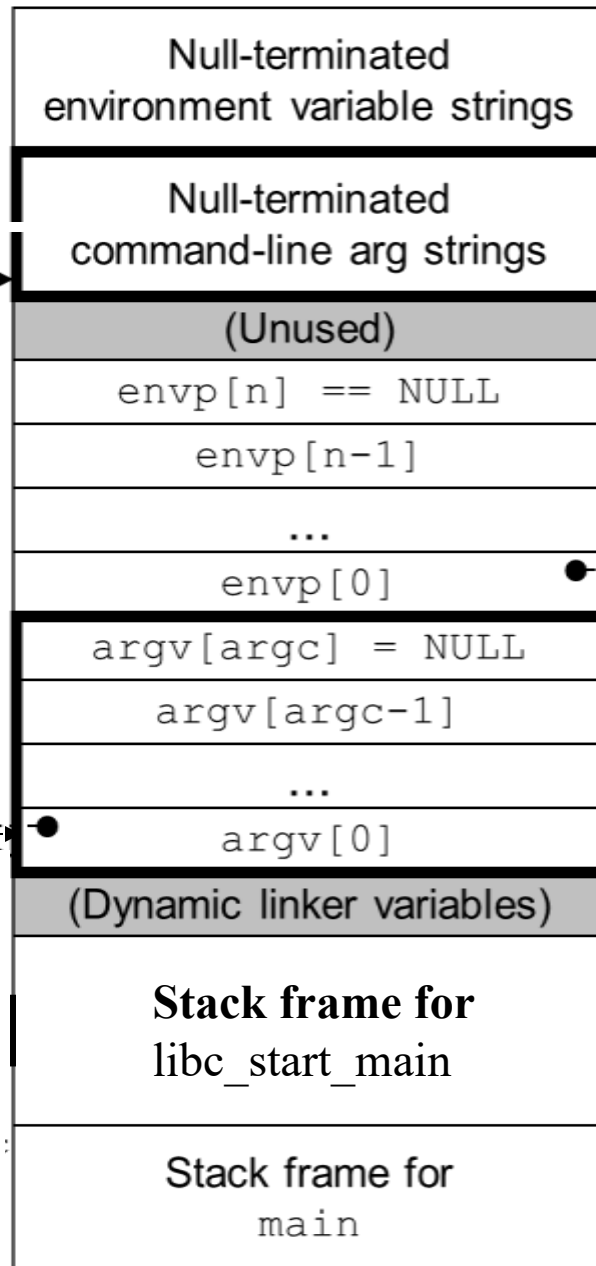
Bottom of stack

```
int main(int argc,
        char **argv,
        char **envp)
```

argv
(in %rsi)

argc
(in %rdi)

0x7fffffffe840



environ
(global var)

envp
(in %rdx)

Top of stack

Loading and Running

```
#include <unistd.h>
```

```
char *getenv(const char *name) ;
```

Returns: ptr to name if exists, NULL if no match.

```
int setenv(const char *name, const char *newvalue,  
           int overwrite) ;
```

Returns: 0 on success, -1 on error.

```
void unsetenv(const char *name) ;
```

Returns: nothing.

Write your Own Shell

Shell

- An interactive application-level program that
 - runs other program on behalf of the user
- Variants: sh, bash, powershell, iterm
- Performs a sequence of read/evaluate steps and terminate
 - Read: reads a command line from the user
 - Evaluate: parses the command line and runs programs on behalf of the user


```
/* The main routine for a simple shell program */
1  #include "csapp.h"
2  #define MAXARGS 128
3
4  /* function prototypes */
5  void eval(char *cmdline);
6  int parseline(const char *cmdline, char **argv);
7  int builtin_command(char **argv);
8
```

```
9  int main()
10 {
11     char cmdline[MAXLINE]; /* command line */
12
13     while (1) {
14         /* read */
15         printf("> ");          // Fgets will add '\0' at the end
16         Fgets(cmdline, MAXLINE, stdin);
17         if (feof(stdin))      // Unix/Linux: Ctrl+D means EOF
18             exit(0);
19
20         /* evaluate */
21         eval(cmdline);
22     }
23 }
```

```
1  /* eval - evaluate a command line */
2  void eval(char *cmdline)
3  {
4      char *argv[MAXARGS]; /* argv for execve() */
5      char buf[MAXLINE]; /* holds modified cmd line */
6      int bg; /* should the job run in bg or fg? */
7      pid_t pid; /* process id */
8
9      strcpy(buf, cmdline);
10     bg = parseline(buf, argv);
11     if (argv[0] == NULL)
12         return; /* ignore empty lines */
13
```

Parsing command line

- Parse the space separated command-line arguments
- Builds the `argv` vector
 - which will eventually be passed to `execve`

```
1  /* parse the cmd line and build the argv array */
2  int parseline(const char *buf, char **argv)
3  {
4      char *delim; /* first space delimiter */
5      int argc; /* number of args */
6      int bg; /* background job? */
7
8      buf[strlen(buf)-1] = ' ';
9      /* replace trailing '\n' with space */
10
12     while (*buf && (*buf == ' ')) /* ignore spaces */
13         buf++;
14
```

```
12  /* build the argv list */
13  argc = 0;
14  while ((delim = strchr(buf, ' '))) {
15      argv[argc++] = buf;
16      *delim = '\0'; /* replace space with '\0' */
17      buf = delim + 1;
18      while (*buf && (*buf == ' ')) /* ignore spaces */
19          buf++;
20  }
21  argv[argc] = NULL; /* set the end of argv list */
22
23  if (argc == 0) /* ignore blank line */
24      return -1;
25
26  /* should the job run in the background? */
27  if ((bg = (*argv[argc-1] == '&')) != 0)
28      argv[--argc] = NULL;
29  return bg;
30 }
```

Foreground and Background

- If the last argument is a "&" character
 - parseline returns 1
 - indicating the program should be executed in the background
(the shell returns to the top of the loop and waits for the next command)
- Otherwise
 - parseline returns 0
 - indicating the program should be run in the foreground
(the shell waits for it to complete)

Parsing command line

- The first argument is assumed to be
 - either the name of a built-in shell command
 - that is interpreted immediately
 - or an executable object file
 - that will be loaded and run in the context of a new child process


```
14  if (!builtin_command(argv)) {
15      if ((pid = Fork()) == 0) { /* execute in child */
16          if (execve(argv[0], argv, environ) < 0) {
17              printf("%s: Command not found.\n", argv[0]);
18              exit(0);
19          }
20      }
21
```

```
14  if (!builtin_command(argv)) {
15      if ((pid = Fork()) == 0) { /* execute in child */
16          if (execve(argv[0], argv, environ) < 0) {
17              printf("%s: Command not found.\n", argv[0]);
18              exit(0);
19          }
20      }
21
```

```
34  /* if 1st arg is a builtin command,
    run it and return true */
35  int builtin_command(char **argv)
36  {
37      if (!strcmp(argv[0], "quit")) /* quit command */
38          exit(0);
39      if (!strcmp(argv[0], "&")) /* ignore singleton & */
40          return 1;
41      return 0; /* not a builtin command */
42  }
```

```
14  if (!builtin_command(argv)) {
15      if ((pid = Fork()) == 0) { /* execute in child */
16          if (execve(argv[0], argv, environ) < 0) {
17              printf("%s: Command not found.\n", argv[0]);
18              exit(0);
19          }
20      }
21
22      /* parent waits for foreground job to terminate */
23      if (!bg) { /* foreground */
24          int status;
25          if (waitpid(pid, &status, 0) < 0)
26              unix_error("waitfg: waitpid error");
27      }
28      else
29          printf("%d %s", pid, cmdline);
30  }
31  return;
32 }
```